



SDD

SOFTWARE DESIGN DESCRIPTIONS

JOHN ALFRED P. SANTOS

C.SoftwareDesignDescriptions

Smart Library Management System (SLMS):

List of Figures

Figure No.	Figure Title	Page
Figure 1	Login Class Diagram (User/Admin Authentication)	10
Figure 2	User Profile Management Class Diagram	12
Figure 3	Seat Data Model Class Diagram	15
Figure 4	Reservation Logic Class Diagram	18
Figure 5	Library Capacity Configuration Class Diagram	20
Figure 6	SmartLib Entity Relationship Diagram (ERD)	25
Figure 7	System Architecture & Intermodule Dependency Diagram	28
Figure 8	Real-time Synchronization Data Flow Diagram (DFD)	30
Figure 9	User Login Data Flow Diagram	31

Figure 10	Seat Selection and Booking Data Flow Diagram	32
Figure 11	Real-time Occupancy Monitoring DFD	33
Figure 12	Automatic Seat Release Logic DFD	34
Figure 13	Admin Capacity Alert System DFD	35
Figure 14	Check-in/Check-out Verification DFD	36
Figure 15	Student Seat Reservation Sequence Diagram	40
Figure 16	Real-time Dashboard Update Sequence Diagram	42
Figure 17	Admin Overcrowding Management Sequence Diagram	44
Figure 18	User Authentication Sequence Diagram	46
Figure 19	Login Page Interface Design	50
Figure 20	Student Home Dashboard (Live Occupancy View)	52
Figure 21	Interactive Library Seat Map (Real-time View)	54

Figure 22	Seat Reservation Confirmation Dialog	55
Figure 23	My Reservations Page Interface	56
Figure 24	User Profile Update Form	58
Figure 25	Admin Live Monitoring Dashboard	60
Figure 26	Admin Seat Management and Status Override Page	62
Figure 27	Library Capacity & Threshold Settings Page	64
Figure 28	Analytics Report: Peak Hours & Occupancy Trends	66
Figure 29	QR Code Check-in Interface	68
Figure 30	System Notifications & Alert Center	70

1. Introduction

This document presents the complete design and development progress of the **The Smart Library Management System (SLMS)** is designed to modernize how libraries operate. Currently, many libraries still face problems with manual record-keeping and unorganized seat allocation. This system aims to solve those issues by digitizing the library's workflow, making it easier for both the staff and the students to manage books and available space.

1.1. Purpose

This document serves as the main guide for the development of the SLMS. The primary goal is to address the problem of **overcrowding** in the library. This SDD explains how features like **Seat Reservation** and **Real-time Monitoring** are designed to control the number of people inside. By following this blueprint, we ensure that the library remains a comfortable and organized place for studying.

1.2. Scope

The system will focus on the following areas:

- **User Platform:** A web and mobile interface where students can log in.
- **Seat Reservation:** A module that allows students to book a specific seat in advance.
- **Real-time Monitoring:** A dashboard that shows the current occupancy and available slots in the library.
- **Admin Management:** Tools for librarians to monitor reservations and manage book records.
- **Exclusions:** The system will not include physical hardware for locking chairs; it will rely on digital check-ins and reservation data.

Table1.AcronymsandDefinitions

Acronym	Definition
ERD	Entity-RelationshipDiagram
SRS	SoftwareRequirementSpecifications
SDD	SoftwareDesignDescriptions
SLMS	Smart Library Management System
UML	UnifiedModellingLanguage

2. References

- [1] R. Stephens, *Beginning Software Engineering*, IN: John Wiley & Sons, Inc., 2015.
 - [2] F. Tsui and O. Karam, *Essentials of Software Engineering*, MA: Jones and Bartlett Publishers, 2011.
 - Websites
 - [3] IEEE Std 830-1998 (Revision of IEEE Std 830-1993)
 - [4] IEEE Std 1058-1998 (Revision and redesignation of IEEE Std 1058.1-1987, incorporating IEEE Std 1058-1998 and IEEE Std 1058a-1998)
- <https://prod.teamgantt.com/gantt>

3. Decomposition description

This section describes the different modules and processes that make up the **Smart Library Management System (SLMS)**. By breaking down the system, we can understand how the

3.1. ModuleDecomposition

The system is divided into three functional modules:

Student/User Module (Mobile & Web): Built using **Flutter**, this is the interface where students interact with the library. Its main functions are to display the live seat map, allow users to book a slot, and show notifications about their reservation status. This module ensures students don't have to walk around the library looking for a spot.

Admin Dashboard Module: This is the control center for library staff. It provides a real-time summary of how many people are currently inside. Librarians can use this to manage the "Maximum Capacity" settings and view analytics on which hours the library is most crowded.

Backend & Database Module (Firestore): This is the "brain" of the system. **Firestore** handles the data storage and the logic for real-time updates. When a seat is booked in the Student Module, the Database instantly sends that update to the Admin Dashboard and all other users.

3.1.1.

AdminModule

Identification	slms/admin/login.php
Type	Form
Purpose	Authenticates the admin user to securely access the system.

Identification	slms/admin/dashboard.php
Type	Page
Purpose	Displays an overview of the system including current library occupancy, crowd status, and overall system activity.

Identification	slms/admin/crowdMonitor.php
Type	Page
Purpose	Displays real-time data from IoT sensors showing the number of students inside the library and determines if it is crowded.

Identification	slms/admin/capacitySettings.php
Type	Page
Purpose	Allows the admin to set and manage the maximum capacity of the library for crowd detection.

Identification	slms/admin/alerts.php
Type	Page
Purpose	Displays system-generated alerts and notifications when the library reaches or exceeds its capacity.

3.1.2

User Module.

Identification	slms/user/login.php
Type	Form
Purpose	Authenticates user (student) access to the system.

Identification	slms/user/register.php
Type	Form
Purpose	Allows the user (student) to create and register their profile in the system.

Identification	slms/user/books.php
Type	Page
Purpose	Displays available books and allows the user to search, view details, and borrow or reserve books.

Identification	slms/user/notifications.php
Type	Page
Purpose	Displays notifications such as due dates, overdue alerts, and reservation updates.

Identification	slms/user/borrowed.php
Type	Page
Purpose	Displays the list of books currently borrowed by the user along with their status.

Identification	slms/user/crowdStatus.php
-----------------------	---------------------------

Type	Page
Purpose	Displays real-time library crowd status (Not Busy, Moderate, or Crowded) using IoT sensor data so users can decide whether to go to the library.

Identification	slms/user/notifications.php
Type	Page
Purpose	Displays notifications such as library crowd alerts, reservation updates, due dates, and overdue reminders.

3.1.3

IOT

Identification	slms/iot/crowdSensor.php
Type	Process
Purpose	Detects the number of people inside the library using sensors (e.g., infrared or camera-based counters) to determine if the library is crowded.

Identification	slms/iot/occupancyMonitor.php
Type	Process
Purpose	Monitors and calculates real-time occupancy levels and compares them to the maximum library capacity.

Identification	slms/iot/crowdStatus.php
Type	Page
Purpose	Displays the current crowd status (e.g., Not Busy, Moderately Busy, Crowded) to users in real-time.

Identification	slms/iot/notificationAlert.php
Type	Process
Purpose	Sends alerts or notifications to students when the library is crowded or when it becomes less busy.

Identification	slms/iot/dataUpdate.php
Type	Process
Purpose	Updates the system database continuously with real-time data from sensors for accurate monitoring

Identification	slms/user/crowdView.php
Type	Page
Purpose	Allows students to check the current library crowd level before going to the library.

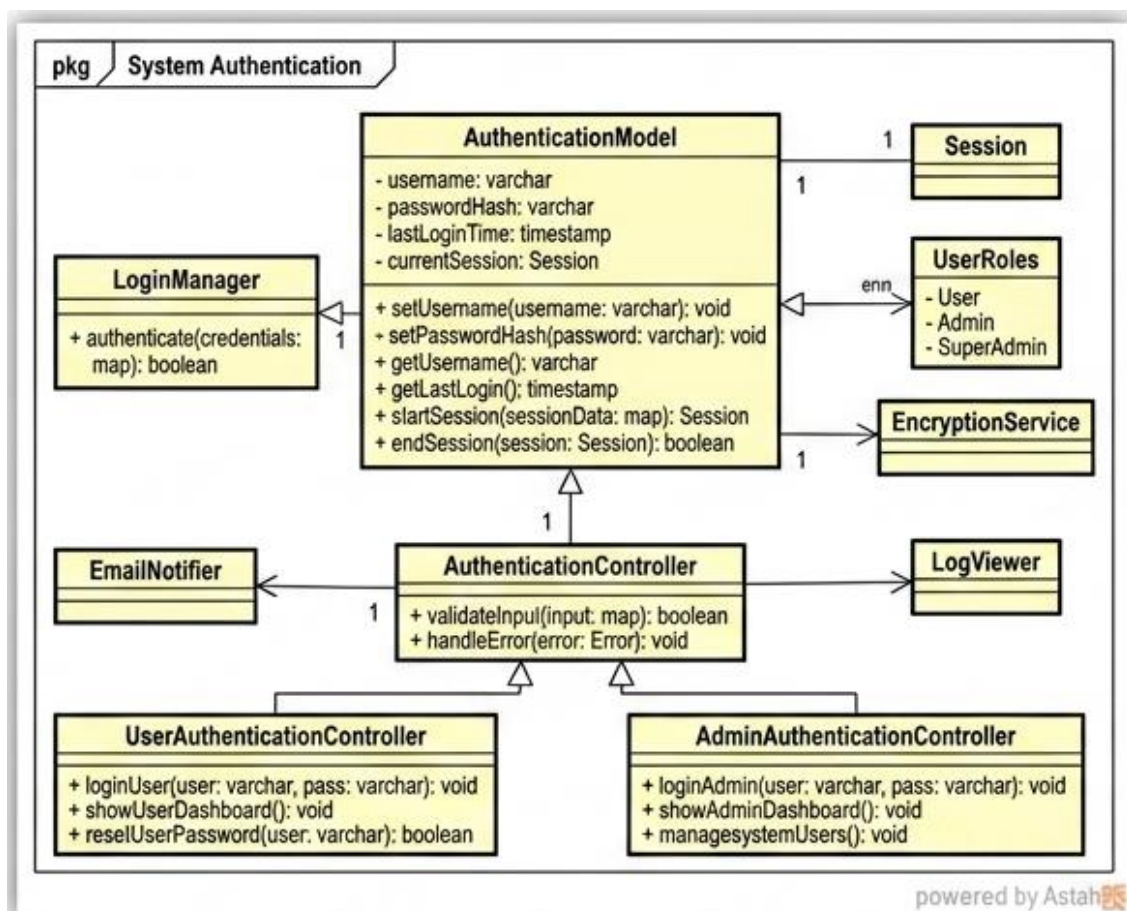
3.2. Concurrent process description

This module displays the real time crowd status of the library based on data collected from IOT sensors. It allows the admin to monitor the number of people inside the library and the correct occupancy level

3.2.1.

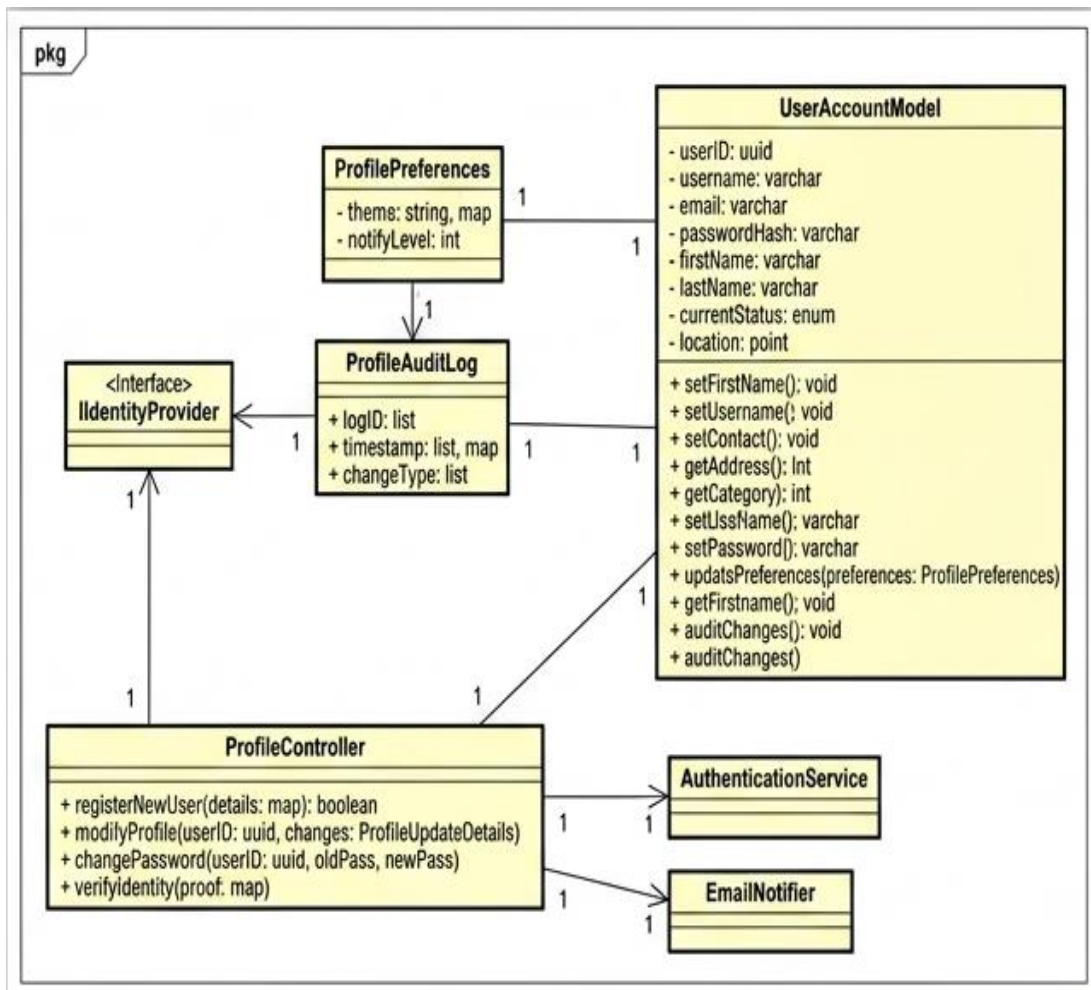
Login

Figure1.LoginClassDiagram



3.2.2. User Profile Management Class Diagram

Figure2 User Profile Management Class Diagram



3.2.3. Seat Data Model Class Diagram

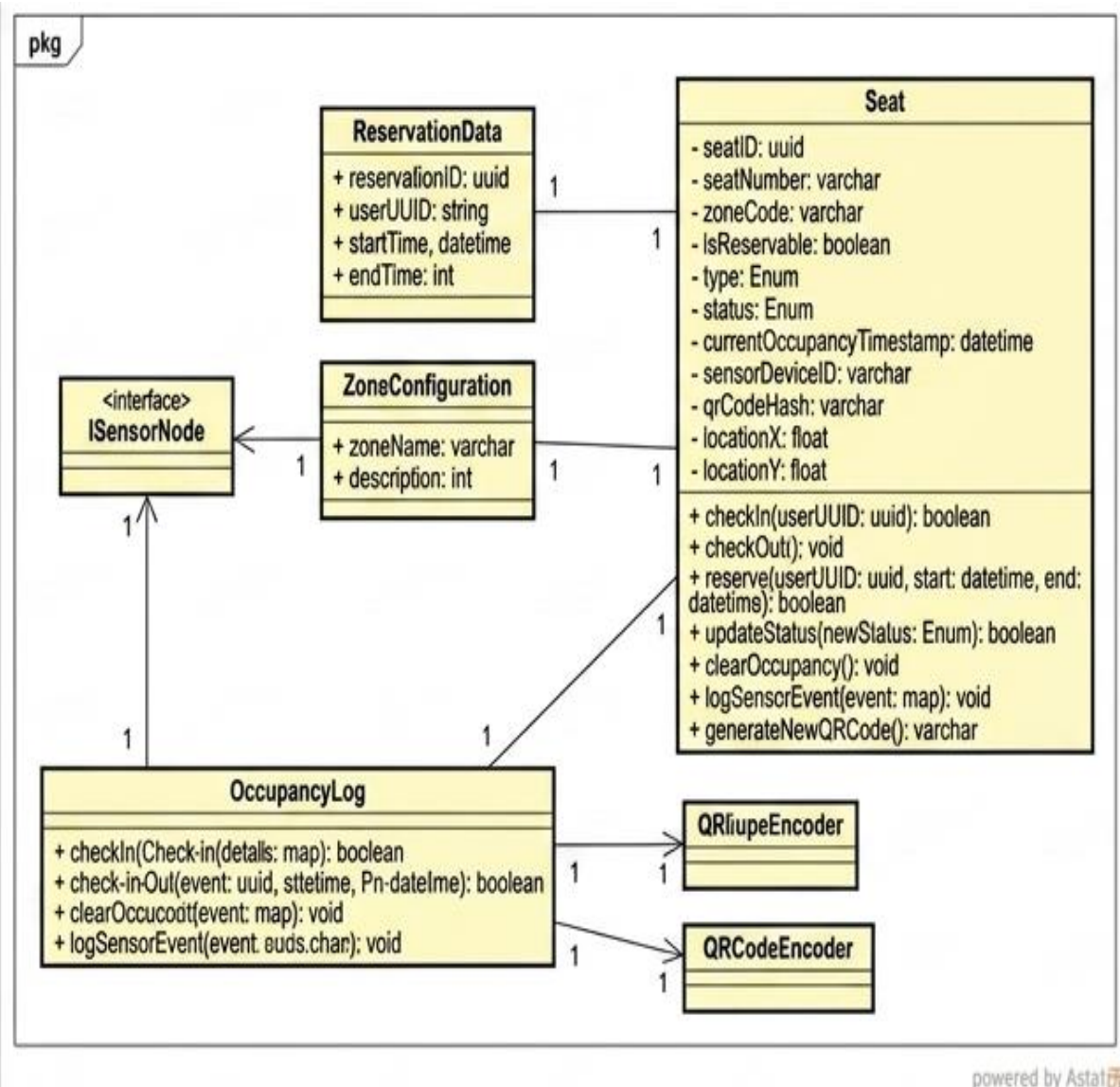


Figure3. Seat Data Model Class Diagram

3.2.4. Reservation Logic Class Diagram

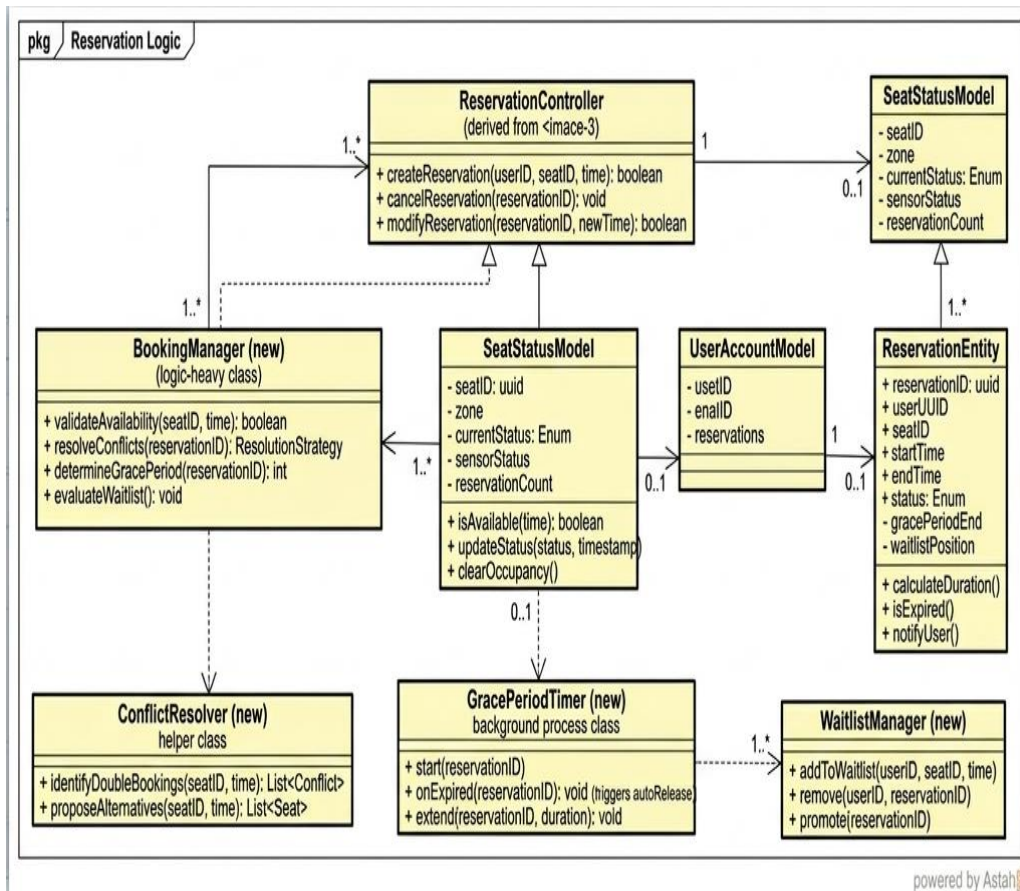


Figure4. Reservation Logic Class Diagram

3.2.5. Library Capacity Configuration Class Diagram

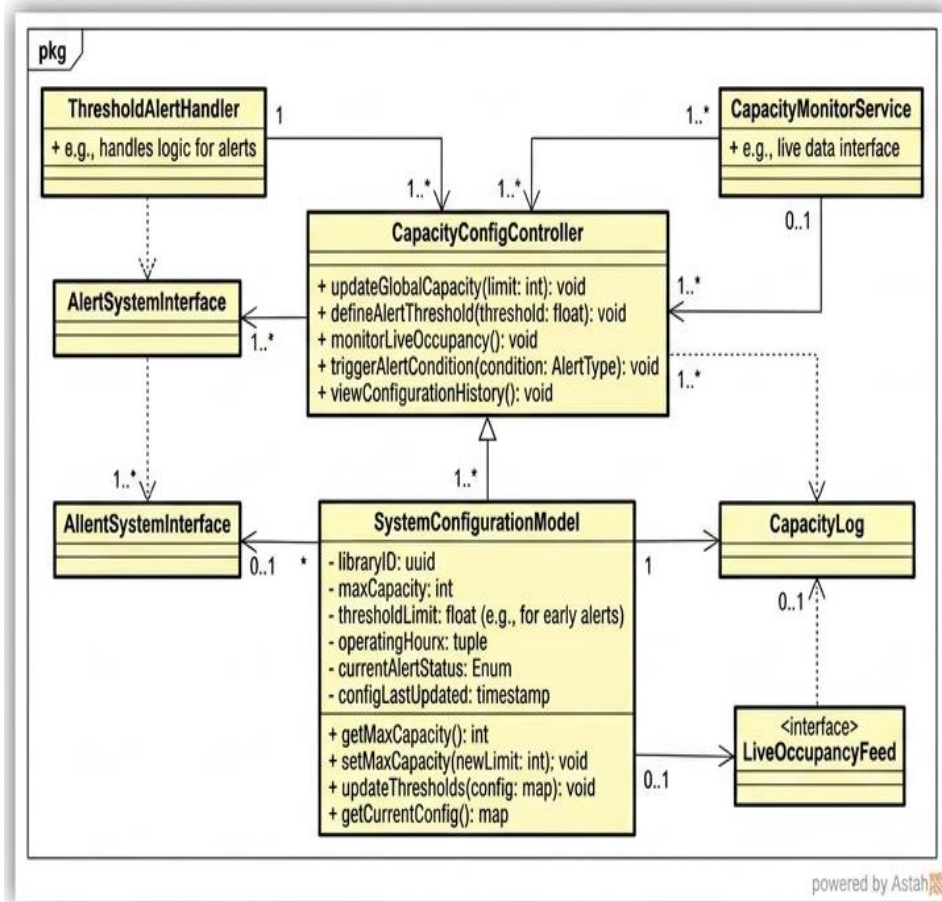


Figure5. Library Capacity Configuration Class Diagram

3.3. Datadecomposition

3.3.1.Data entity: Library Admin Information

```
CREATE TABLE `admin_info` (  
  
  `admin_id` INT(11) NOT NULL AUTO_INCREMENT,  
  
  `firstname` VARCHAR(50) NOT NULL,  
  
  `middlename` VARCHAR(50) NOT NULL,  
  
  `lastname` VARCHAR(50) NOT NULL,  
  
  `username` VARCHAR(50) NOT NULL,  
  
  `password` VARCHAR(50) NOT NULL,  
  
  `position` VARCHAR(20) NOT NULL,  
  
  `admin_status` VARCHAR(10) NOT NULL,  
  
  PRIMARY KEY (`admin_id`)  
  
);
```

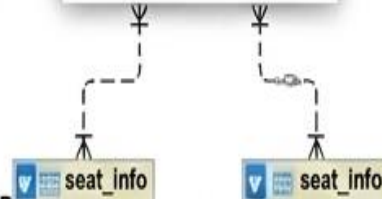
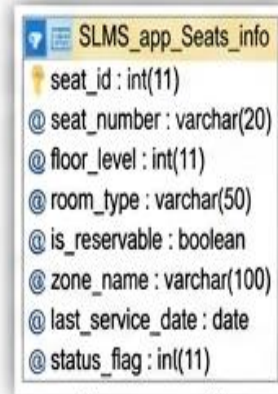
3.3.2.Data Entity:Student Information

```
CREATE TABLE `client_info` (  
  
  `client_id` INT(11) NOT NULL AUTO_INCREMENT,  
  
  `firstname` VARCHAR(50) NOT NULL,  
  
  `middlename` VARCHAR(50) NOT NULL,  
  
  `lastname` VARCHAR(50) NOT NULL,  
  
  `address` VARCHAR(50) NOT NULL,  
  
  `username` VARCHAR(50) NOT NULL,  
  
  `password` VARCHAR(50) NOT NULL,  
  
  `clientCategory_ID` INT(11) NOT NULL,  
  
  `contactNumber` VARCHAR(20) NOT NULL,  
  
  `brgyID` INT(11) NOT NULL,  
  
  PRIMARY KEY (`client_id`)  
  
);
```

3.3.3.Dataentity:SeatInfo

--Table structure for table 'seats_info'

```
CREATE TABLE 'seats_info' (
  'seat_id' int(11) NOT NULL
  auto_increment,
  'seat_number' varchar(20) NOT NULL,
  'floor_level' int(11) NOT NULL,
  'room_type' varchar(50),
  'is_reservable' boolean NOT NULL,
  'zone_name' varchar(100),
  'last_service_date' date,
  'status_flag' int(11) NOT NULL,
  PRIMARY KEY ('seat_id')
);
```



3.3.4.Data entity:Updated SQL Table Structure (SLMS)

--
 -- Table structure for table 'seat_schedule'
 -- This handles the timing and location for seat availability

```
CREATE TABLE `seat_schedule` (
  `scheduleId` int(11) NOT NULL AUTO_INCREMENT,
  `seat_id` int(11) NOT NULL,
  `zone_id` int(11) NOT NULL,
  `available_date` date NOT NULL,
  `start_time` time NOT NULL,
  `end_time` time NOT NULL,
  `status` varchar(20) DEFAULT 'Available',
  PRIMARY KEY (`scheduleId`)
);
```

3.3.5.Dataentity:SeatBookingSchedule

```
--
-- Table structure for table 'seat_booking_schedule'
--
```

```
CREATE TABLE 'seat_booking_schedule' (
  'booking_id' int(11) NOT NULL AUTO_INCREMENT,
  'seat_number' varchar(500) NOT NULL,
  'zone_id' int(11) NOT NULL,
  'library_branch_id' int(11) NOT NULL,
  'checkinTime' time NOT NULL,
  'checkoutTime' time NOT NULL,
  'bookingDate' date NOT NULL,
  'user_type' int(11) NOT NULL,
  'lockStatus' int(11) NOT NULL,
  PRIMARY KEY ('booking_id')
);
```



SLMS_app_SeatBooking_schedule

🔑	booking_id : int(11)	
#	seat_number : varchar(500)	
#	zone_id : int(11)	
#	library_branch_id : int(11)	
@	checkinTime : time	
@	checkoutTime : time	
@	bookingDate : date	
@	user_type : int(11)	
@	lockStatus : int(11)	

powered by SLMS

3.3.6.Data Entity: SeatReservation

```
--
-- Table structure for table 'seat_reservation'
--
```

```
CREATE TABLE `seat_reservation` (
  `reservation_id` int(11) NOT NULL AUTO_INCREMENT,
  `student_id` int(11) NOT NULL,
  `seat_id` int(11) NOT NULL,
  `zone_id` int(11) NOT NULL,
  `status` int(11) NOT NULL, -- (Halimbawa: 1=Confirmed, 2=Pending,
  0=Cancelled)
  PRIMARY KEY (`reservation_id`)
);
```



SLMS_app_SeatReservation
(simplified)

🔑	reservation_id : int(11)	1
#	student_id : int(11)	1
#	seat_id : int(11)	1
#	zone_id : int(11)	1
#	status : int(11)	1

powered by SLMS

3.3.7.Dataentity:Data Entity: Library Policy

```
CREATE TABLE `library_policy` (
  `policy_id` int(11) NOT NULL AUTO_INCREMENT,
  `policy_title` varchar(500) NOT NULL,
  `policy_desc` varchar(5000) NOT NULL,
  PRIMARY KEY (`policy_id`)
);
```

SLMS_app_LibraryPolicy (id: library_policy_01)		
policy_id : int polic_desc: varchar(255) policy_type : char(1) status : saar(1) priority : bool		
[policy_id]	[policy_title] [policy_title]	[policy_desc]
1	Max Stay	Maximum stay of 4 hours to reduce overcrowding.
2	No Seat Reservation	No reservation of seats using personal items (e.g., bags).
powered by SLMS_CORE		

3.3.8.Data entity:Data Entity: Zone Category

```
CREATE TABLE `zone_category` (
  `zone_id` int(11) NOT NULL AUTO_INCREMENT,
  `zone_name` varchar(500) NOT NULL,
  PRIMARY KEY (`zone_id`)
);
```

SLMS_app_ZoneCategory (id: zone_category_01)			
zone_id : int zone_name : varchar(500) area_capacity : int current_occupancy : int			
[zone_id]	[zone_name] [zone_title]	area_capacity : int	current_occu ancy : int
1	Quiet Zone	20	18
2	Group Study Zone	40	12
3	Social Zone	30	5
powered by SLMS_CORE			

3.3.9.Data Entity: Zone Section

```
CREATE TABLE `zone_section` (  
  `section_id` int(11) NOT NULL AUTO_INCREMENT,  
  `section_name` varchar(500) NOT NULL,  
  `zone_id` int(11) NOT NULL,  
  PRIMARY KEY (`section_id`)  
);
```

SLMS_app_ZoneSection (id: zone_section_01)		
section_id : int typdn : varchar(255) zone_id status : int		
[section_id] [section_id]	[section_name] [section_name]	[zone_id] [zone_id]
1	Table Area A 	1 (link to Quiet Zone)
2	Cubicle Row B 	1 (link to Quiet Zone)
3	Collaborative pod C 	2 (to Group Study Zone)
powered by SLMS_CORE		

3.3.10. Data Entity: Seat Allocation

```
CREATE TABLE `seat_allocation` (
  `allocation_id` int(11) NOT NULL AUTO_INCREMENT,
  `section_id` int(11) NOT NULL,
  `user_id` int(11) NOT NULL,
  `status` int(11) NOT NULL,
  PRIMARY KEY (`allocation_id`)
);
```

SLMS_app_SeatAllocation (id: seat_allocation_01)				
allocation_id : int typdn : varchar(255) user_id status : int				
[allocation_id] [allocation_id]	[section_id] [section_id]	[user_id] [user_id]	[status] [status]	
1	1 (Table Area A) 	User 101	1 (Occupied)	
2	2 (Cubicle Row B) 	User 105	1 (Occupied)	
3	3 (Collab. pod C) 	null	0 (Vacant)	
powered by SLMS_CORE				

3.3.11. Data Entity: Occupancy Validation

```
--
CREATE TABLE `occupancy_validation` (
  `validation_id` int(11) NOT NULL AUTO_INCREMENT,
  `user_id` int(11) NOT NULL,
  `schedule_id` int(11) NOT NULL,
  PRIMARY KEY (`validation_id`)
);
```

SLMS_app_OccupancyValidation (id: occupancy_validation_01)			
validation_id : int typdn : varchar(255) user_id_status : int			
[validation_id] [validation_id]	[user_id] [user_id]	[schedule_id] [schedule_id]	
1	User 101 	Sched_01 (Time Slot 1)	
2	User 105 	Sched_02 (Time Slot 2)	
3	User 110 	Sched_01 (Time Slot 1)	
powered by SLMS_CORE			

3.3.12. Data Entity: User Category

```
--
CREATE TABLE `user_category` (
  `userCategory_ID` int(11) NOT
NULL AUTO_INCREMENT,
  `userCategory_name` varchar(20)
NOT NULL,
  PRIMARY KEY (`userCategory_ID`)
);
```

SLMS_app_UserCategory (id: user_category_01)	
userCategory_ID : int typdn : varchar(255) user_id status : int	
[userCategory_ID] [userCategory_ID]	[userCategory_name] [userCategory_name]
1	Student 
2	Faculty 
3	Guest 
powered by SLMS_CORE	

3.3.13. Data Entity: Space Feedback

```
--
--Tablestructurefortable'rating' --
--
CREATETABLE'rating'(
  'ratingID' int(11) NOT NULL
auto_increment,
  'questionE'varchar(1000)NOTNULL,
  'questionH'varchar(1000)NOTNULL,
  PRIMARY KEY ('ratingID'),
);
```

SLMS_app_SpaceFeedback (id: space_feedback_01)		
feedback_ID : int typdn : varchar(255) user_id status : int		
[feedbackID] [feedbackID]	[question_en] [question_en]	[question_local] [question_local]
1	Is the space too crowded? 	Masyado bang masikip ang espasyo?
2	Is the noise level acceptable? 	Maingay ba sa Quiet Zone?
3	Rate the temperature.	Kumportable ba ang temperatura?
powered by SLMS_CORE		

3.3.14. Data Entity: Occupancy Report

--

```
-CREATE TABLE `report_questions` (  
  `reportQ_ID` int(11) NOT NULL  
  AUTO_INCREMENT,  
  `report_text` varchar(1000) NOT  
  NULL,  
  PRIMARY KEY (`reportQ_ID`)  
);
```

SLMS_app_OccupancyReport (id: occupancy_report_01)					
occupancy_id : int typdn : varchar(255) user_id status : int					
[reportID] [reportID]	[allocation_id] [allocation_id]	[userID] [userID]	[comment_1] [comment_1]	[comment_2] [comment_2]	[comment_3] [comment_3]
1	1 (linking to Table Area A)	User 101	Chair is broken 	No power outlet	Area is very loud
2	2 (linking to Cubicle Row B)	User 105	null	Table is dirty 	Monitor is not working
3	null	User 110	No available seat in Quiet Zone	null	null
powered by SLMS_CORE					

3.3.15. Data Entity: Report Questions s

--

```
CREATE TABLE `report_questions` (  
  `reportQ_ID` int(11) NOT NULL AUTO_INCREMENT,  
  `report_text` varchar(1000) NOT NULL,  
  PRIMARY KEY (`reportQ_ID`)  
);
```

SLMS_app_ReportQuestions (id: report_questions_01)	
reportQ_ID : int typdn : varchar(255) user_id status : int	
[reportQ_ID] [reportQ_ID]	[report_text] [report_text]
1	Predefined option: Equipment Issue 
2	Predefined option: Environment Issue 
3	Predefined option: Occupancy Status 
powered by SLMS_CORE	


```
--
--Tablestructurefortable'ratingclient'--
--
CREATETABLE'ratingclient'(
    'ratingClientID'    int(11)    NOT
                        NULL
auto_increment,
    'ratingID' int(11) NOT NULL,
    'clientID' int(11) NOT NULL,
    'trainingSchedule' int(11) NOT NULL,
    'rate' int(11) NOT NULL,
    PRIMARYKEY('ratingClientID'),
);
```

SLMS_app_SpaceQualityRating (id: space_quality_rating_01)				
qualityRating_Id: int varchar(255) rule_type: char(1) onatatus : bool priority: int active: int				
[qualityRating_ID] [quality desc]	[feedbackID] [feedbackID]	[userID] [userID]	[allocation_id] [allocation_id]	[rate_score] [rate_score]
 1	1	User 101	1	4 (Good)
 2	2	User 105	2	2 (Crowded)
 3	1	User 110	null	5 (Perfect)
powered by SLMS_CORE				

3.3.17. **Data Entity: Zone Location**

```
--
--Tablestructurefortable'address'--
--
CREATETABLE'address' (
    'address_id' int(11)NOTNULLauto_increment,
    'address' varchar(100) NOT NULL,
    PRIMARYKEY('address_id'),
);
```

SLMS_app_ZoneLocation (id: zone_location_01)			
location_id : int lon_desc : varchar(255) lon type : char(1) icon_res : varchar(50) icon_ref: bool priority : int			
	[location_id] [location_id]	[location_desc] [location_desc]	
	1	2nd Floor, West Wing	
	2	Ground Floor, Main Hall	
	3	Basement Level, Stack Area	
powered by SLMS_CORE			

3.3.18. Data Entity: Occupancy Masterlist

--

```
CREATE TABLE `occupancy_masterlist` (
  `masterlistID` int(11) NOT NULL AUTO_INCREMENT,
  `section_id` int(11) NOT NULL, -- Link sa Zone Section
  `schedule_id` int(11) NOT NULL,
  `userID` int(11) NOT NULL,
  PRIMARY KEY (`masterlistID`)
);
```

SLMS_app_OccupancyMasterlist (id: occupancy_masterlist_01)				
masterlistID : int rule_desc: varchar(255) rule_type: char(1) status : bool priority : int active : int				
	[masterlistID] [masterlistID]	[section_id] [section_id]	[schedule_id] [schedule_id]	[userID] [userID]
	1	1	Sched_01	User 101 
	2	2	Sched_02	User 105 
	3	null	null	User 110 
				powered by SLMS_CORE

3.3.19. Data Entity: Proof of Occupancy

--

```
CREATE TABLE `occupancy_proof` (
  `proof_id` int(11) NOT NULL AUTO_INCREMENT,
  `file_name` varchar(200) NOT NULL,
  `validation_id` int(11) NOT NULL, -- Link sa Occupancy Validation
  PRIMARY KEY (`proof_id`)
);
```

SLMS_app_ProofOfOccupancy (id: occupancy_proof_01)			
proof_id : int file_name : varchar(255) rile_type : char(1) : bool priority : int active : int			
	[proof_id] [proof_id]	[file_name] [file_name]	[validation_id] [validation_id]
	1	 seat_101_checkin.jpg	1
	2	 id_card_user105.png	2
	3	 monitor_report.pdf	null
			powered by SLMS_CORE

3.4Entity-
RelationshipDiagram

Figure 6. SmartLib Entity Relationship Diagram (ERD)

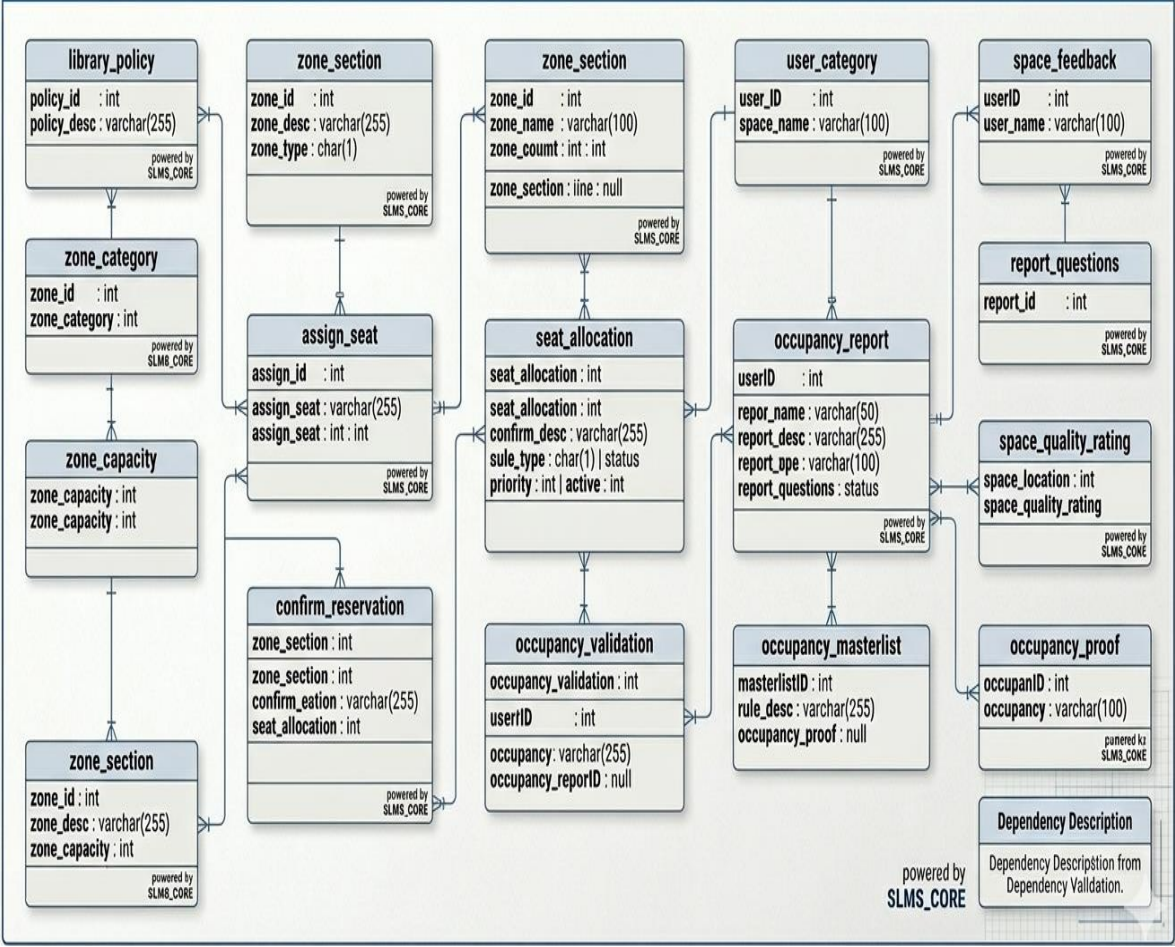


Figure 6. SLMS Entity Relationship Diagram (ERD)

4. Dependency description

4.1. Intermodule dependencies

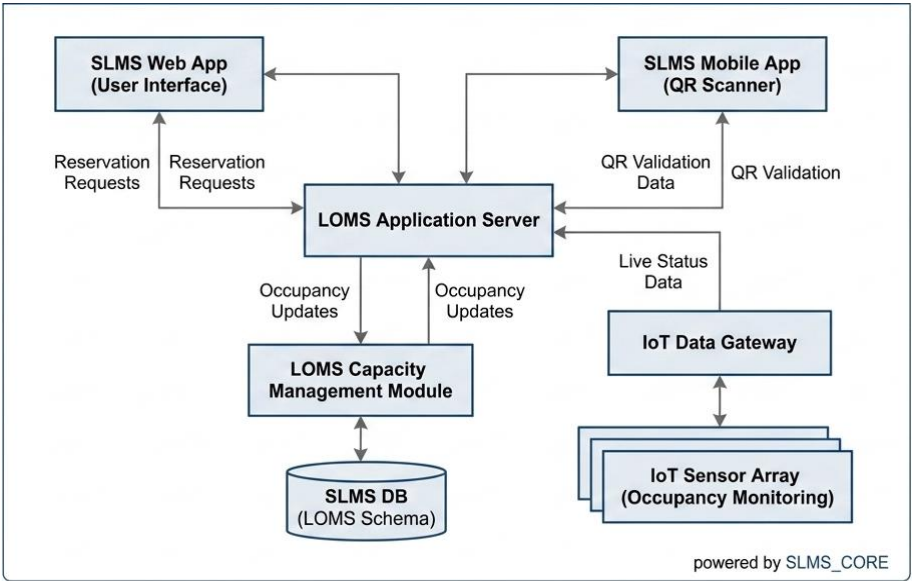


Figure 7. System Architecture & Intermodule Dependency Diagram

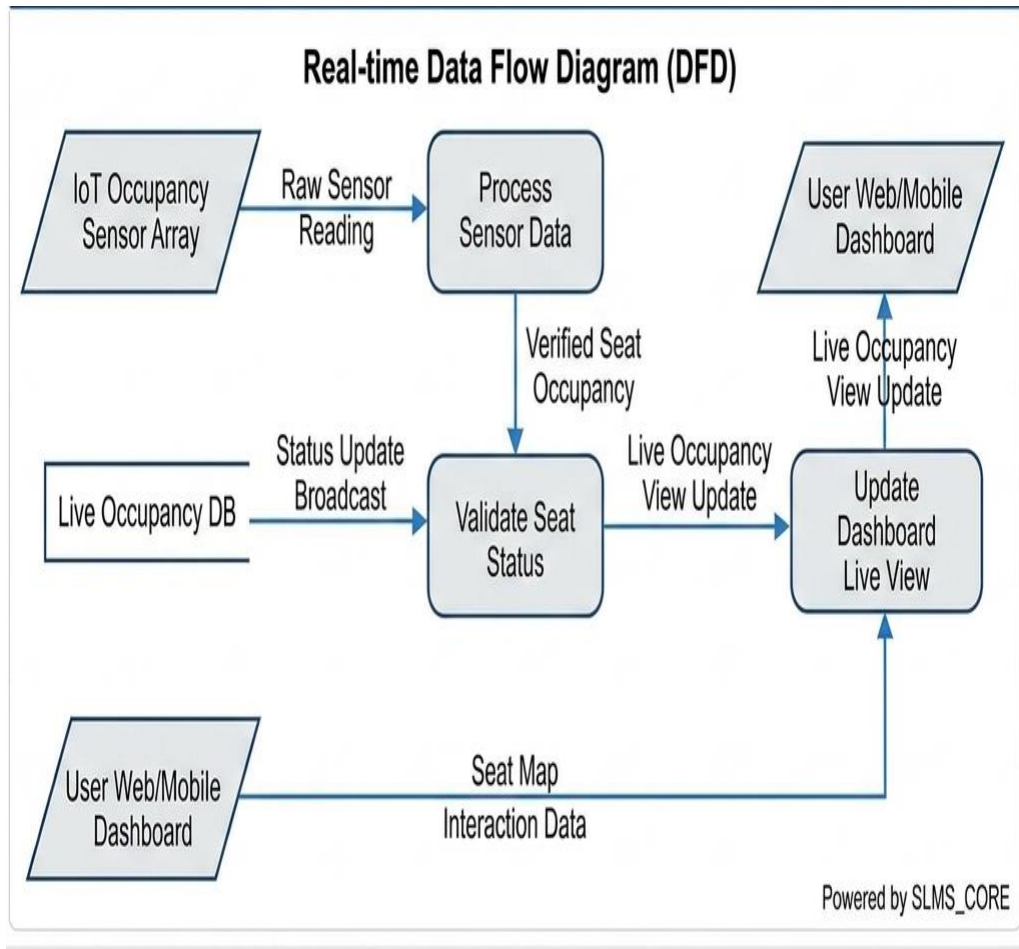


Figure 8. Real-time Synchronization Data Flow Diagram (DFD)

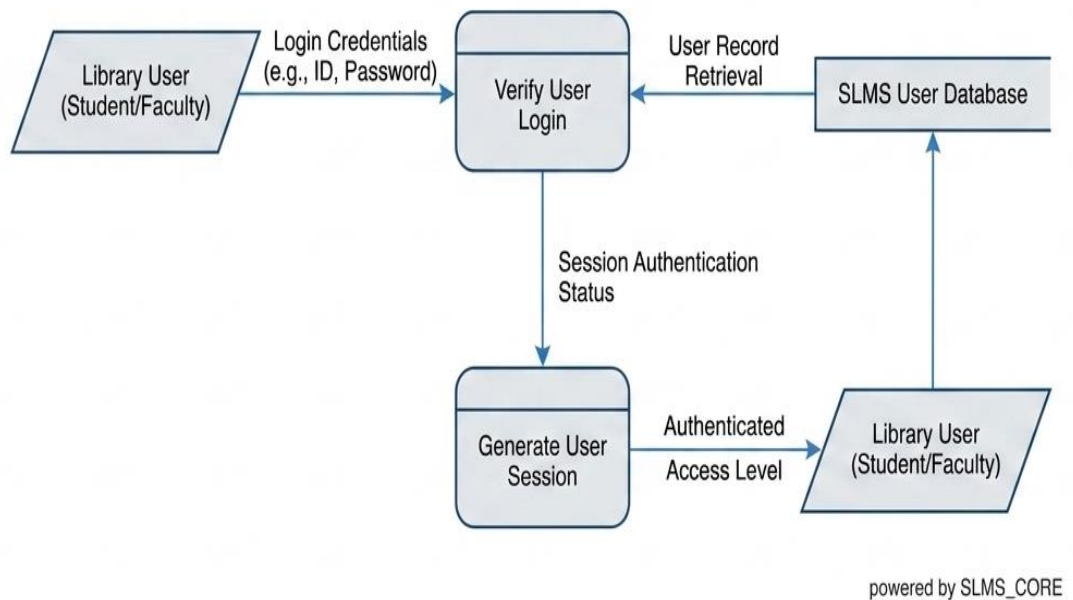


Figure 9. User Login Data Flow Diagram

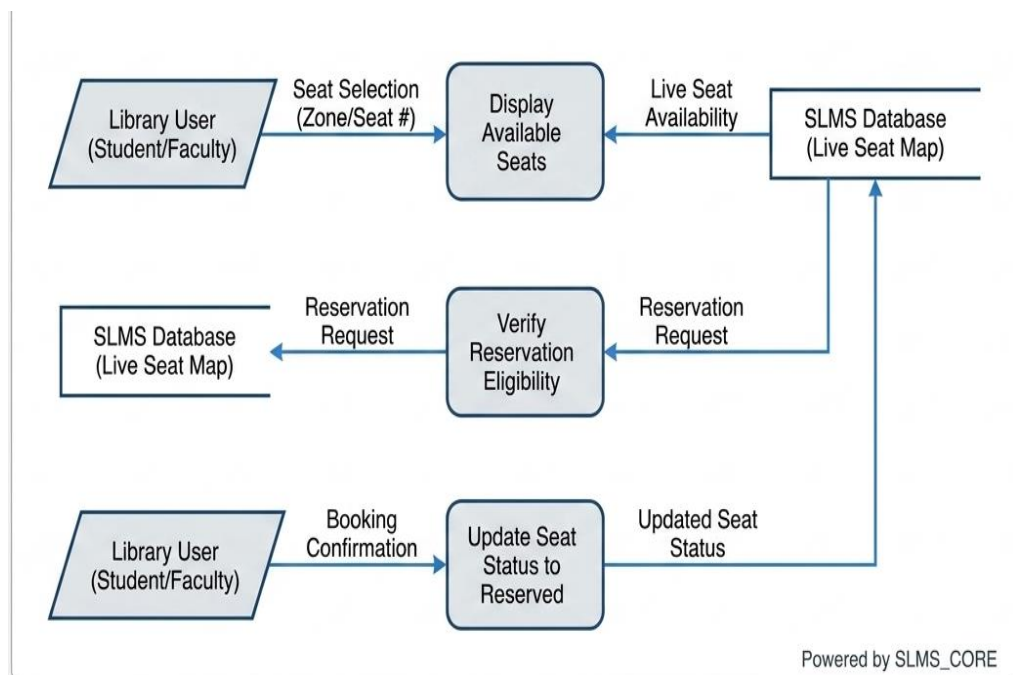


Figure 10. Seat Selection and Booking Data Flow Diagram

4.2. Data dependencies

The **Data Flow Diagrams (DFDs)** depict the dependencies between data within the **Smart Library Management System (SLMS)**. These diagrams illustrate the flow of information between library users, sensors, and the central database to manage occupancy and prevent overcrowding. The succeeding figures show these data transitions.

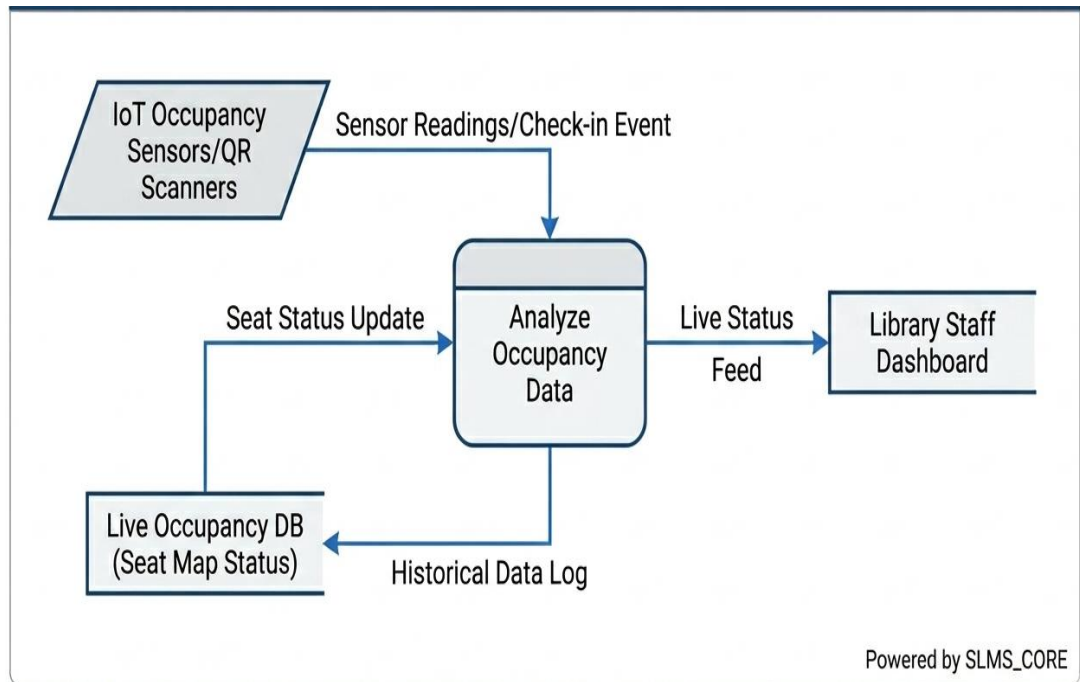


Figure 11. Real-time Occupancy Monitoring DFD

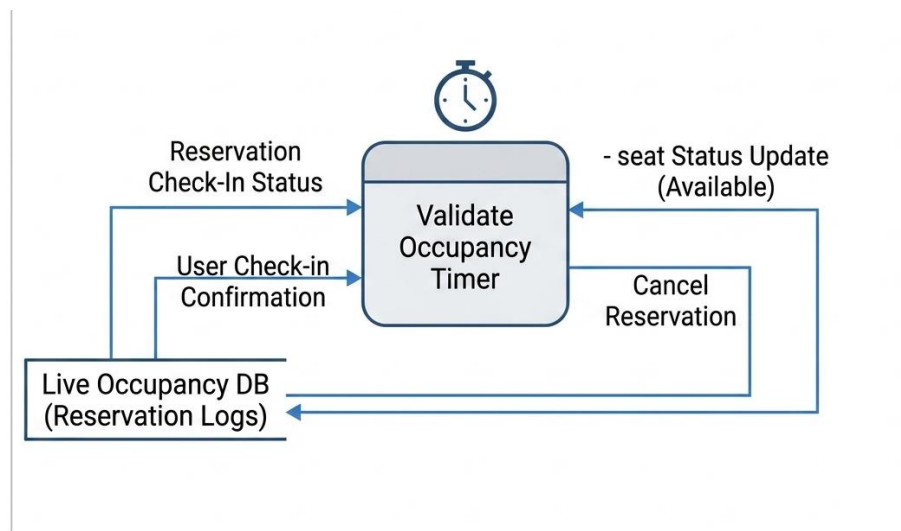


Figure 12. Automatic Seat Release Logic DFD

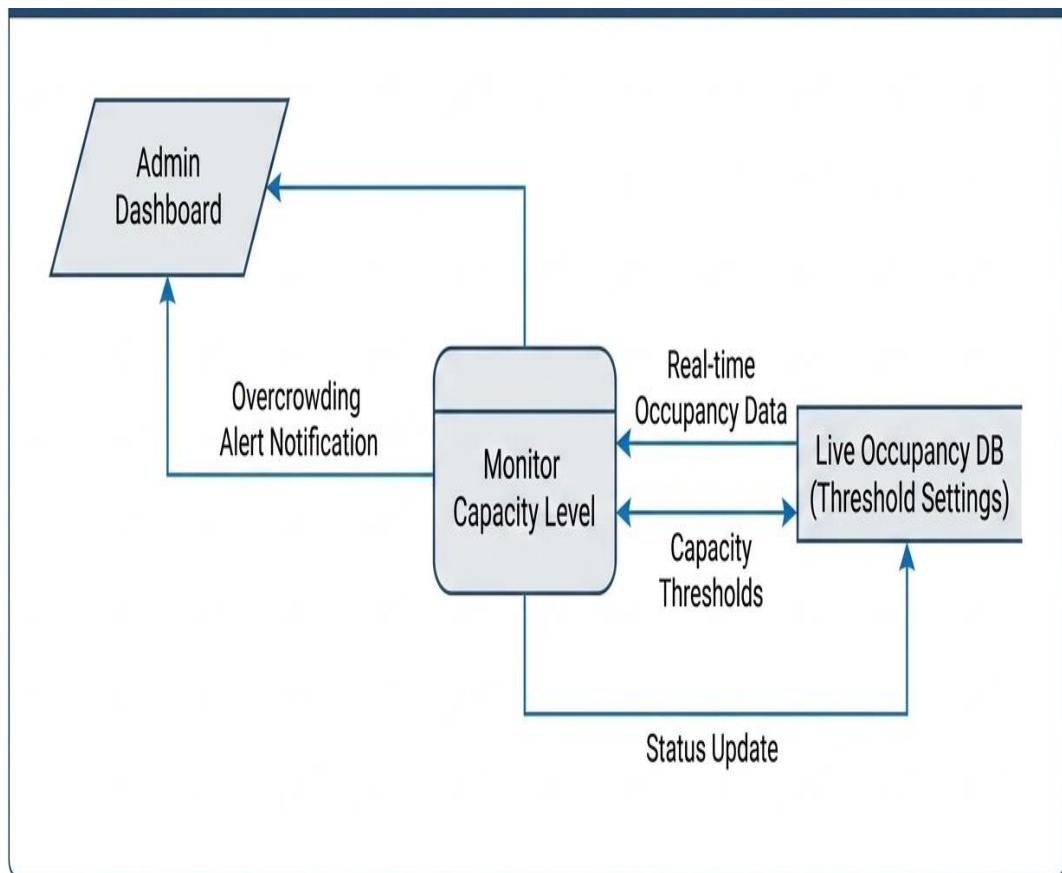


Figure 13. Admin Capacity Alert System DFD

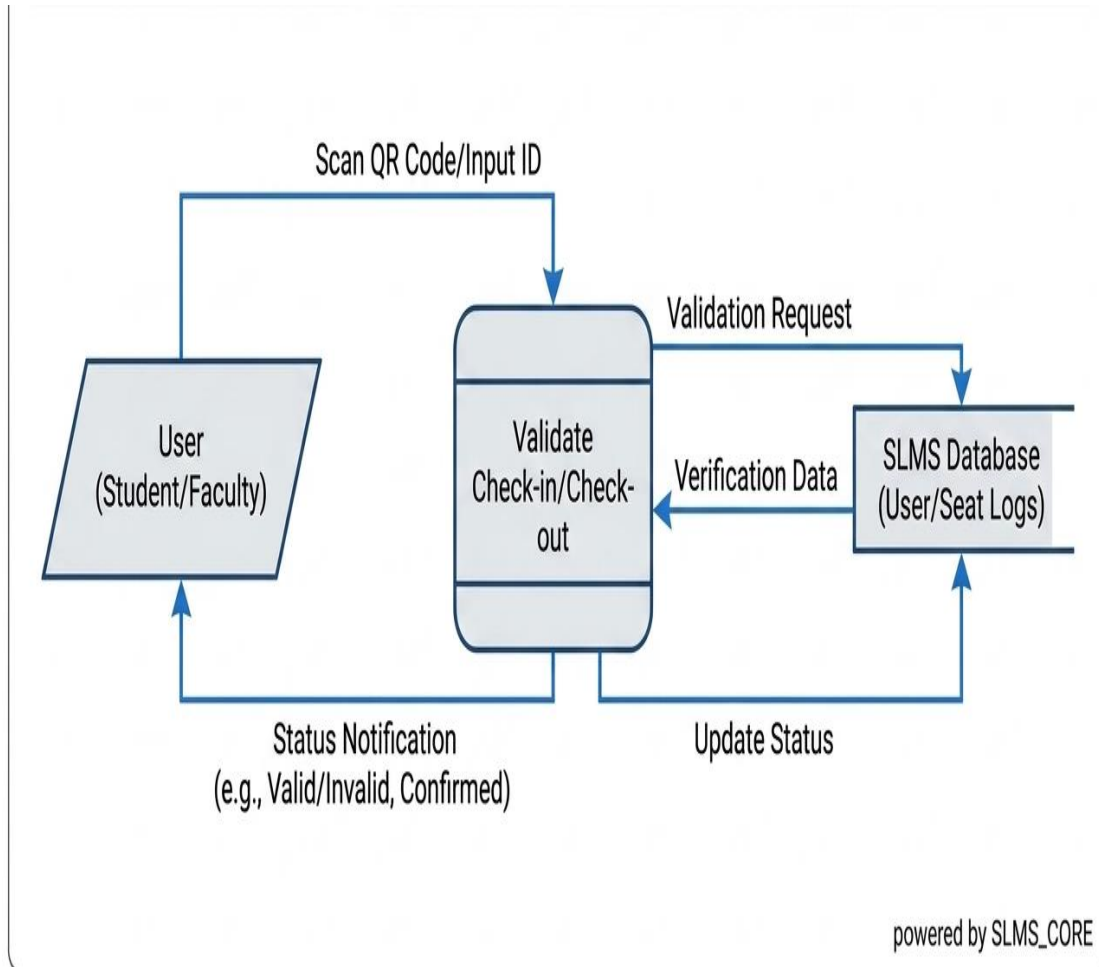


Figure 14. Check-in/Check-out Verification DFD

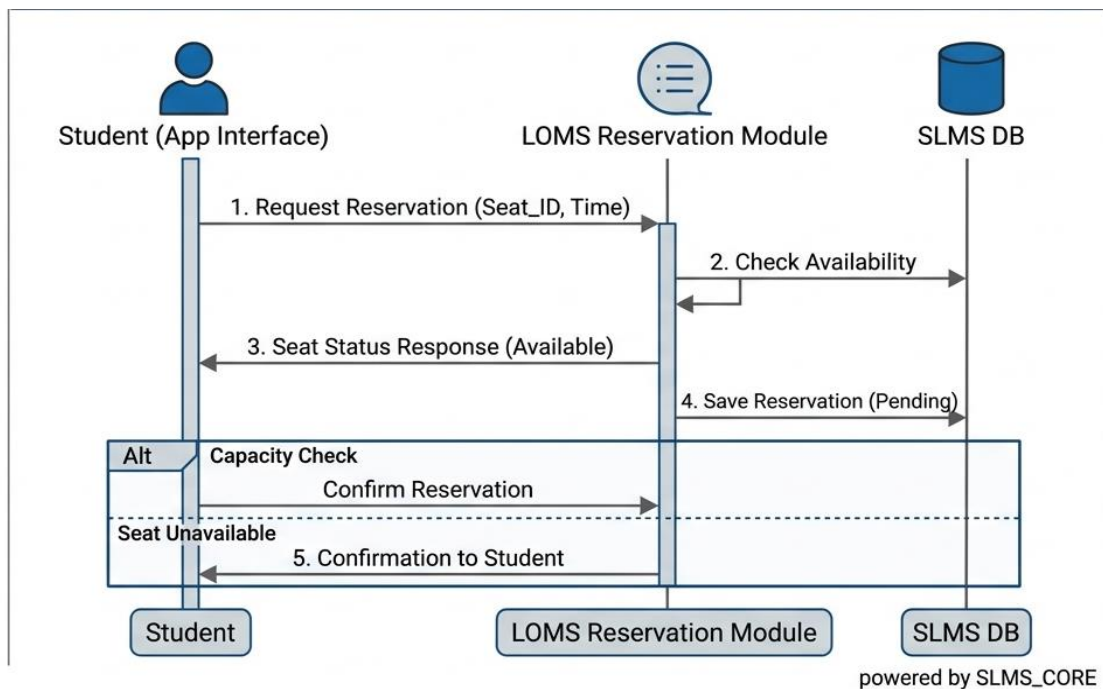


Figure 15. Student Seat Reservation Sequence Diagram

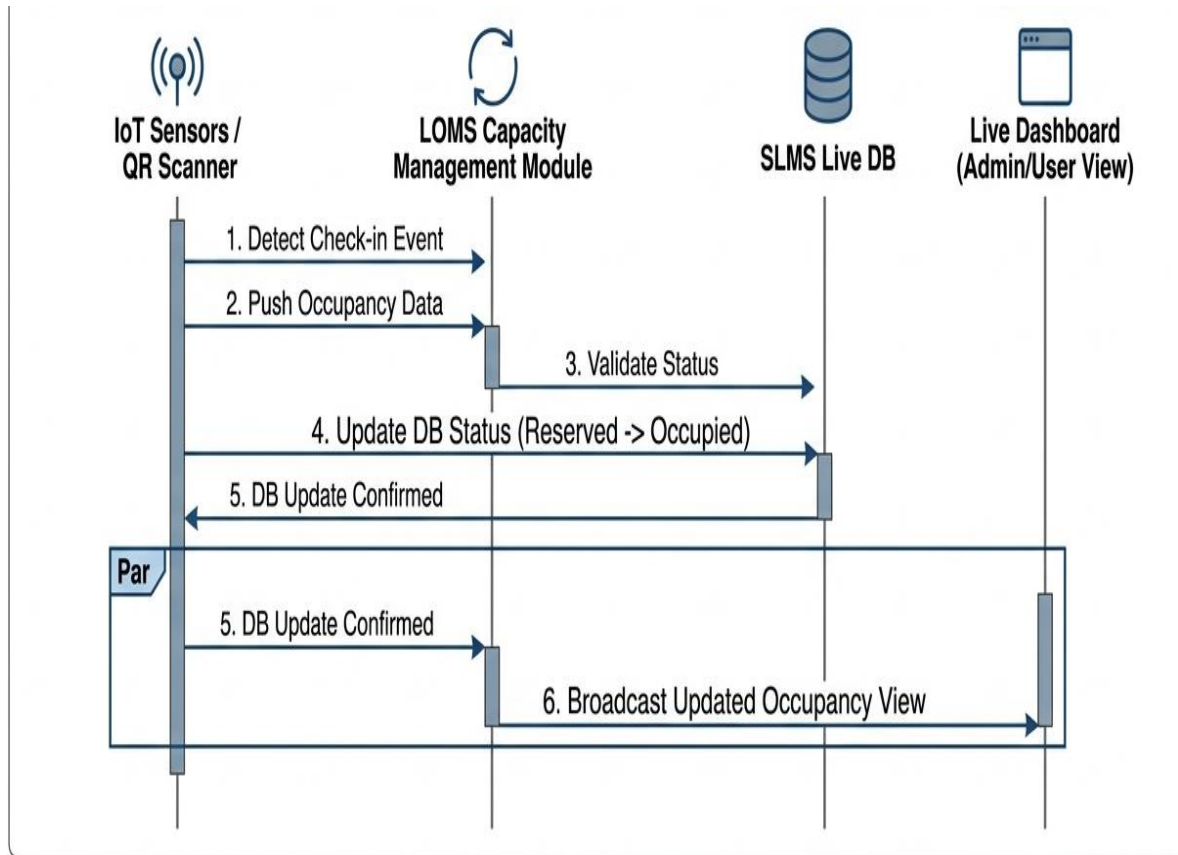


Figure 16. Real-time Dashboard Update Sequence Diagram

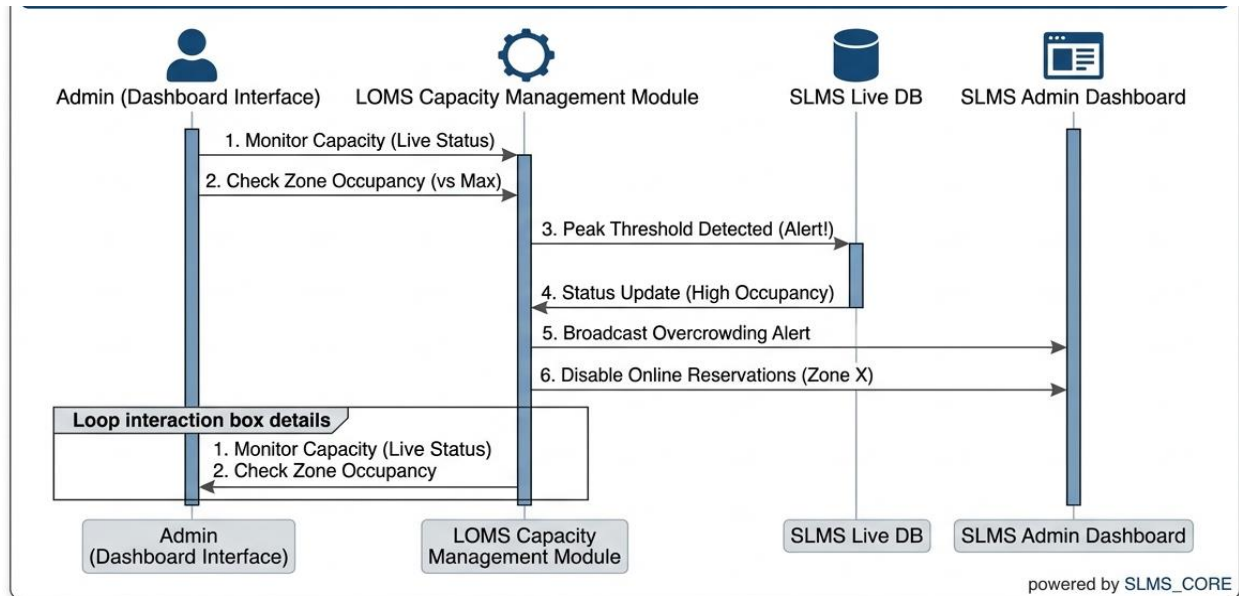


Figure 17. Admin Overcrowding Management Sequence Diagram

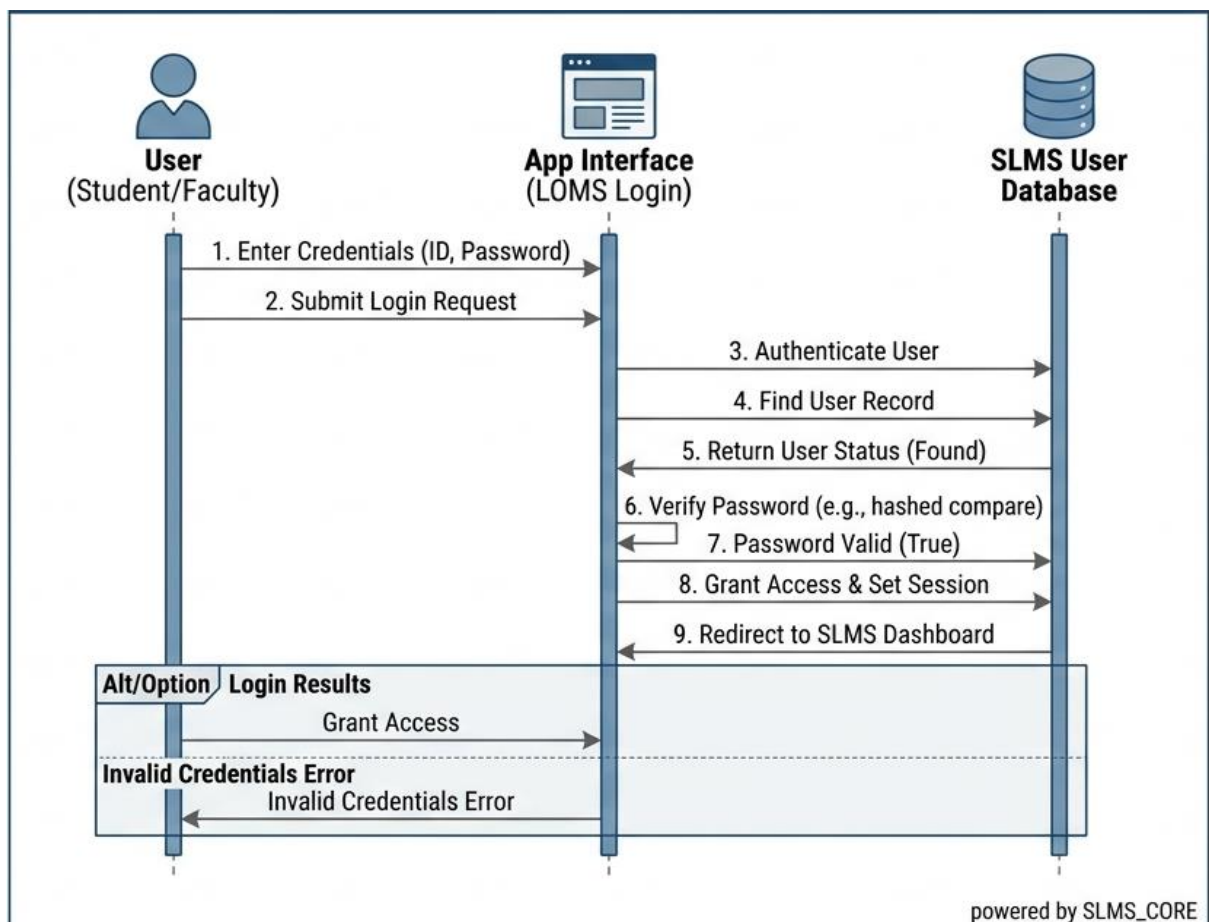
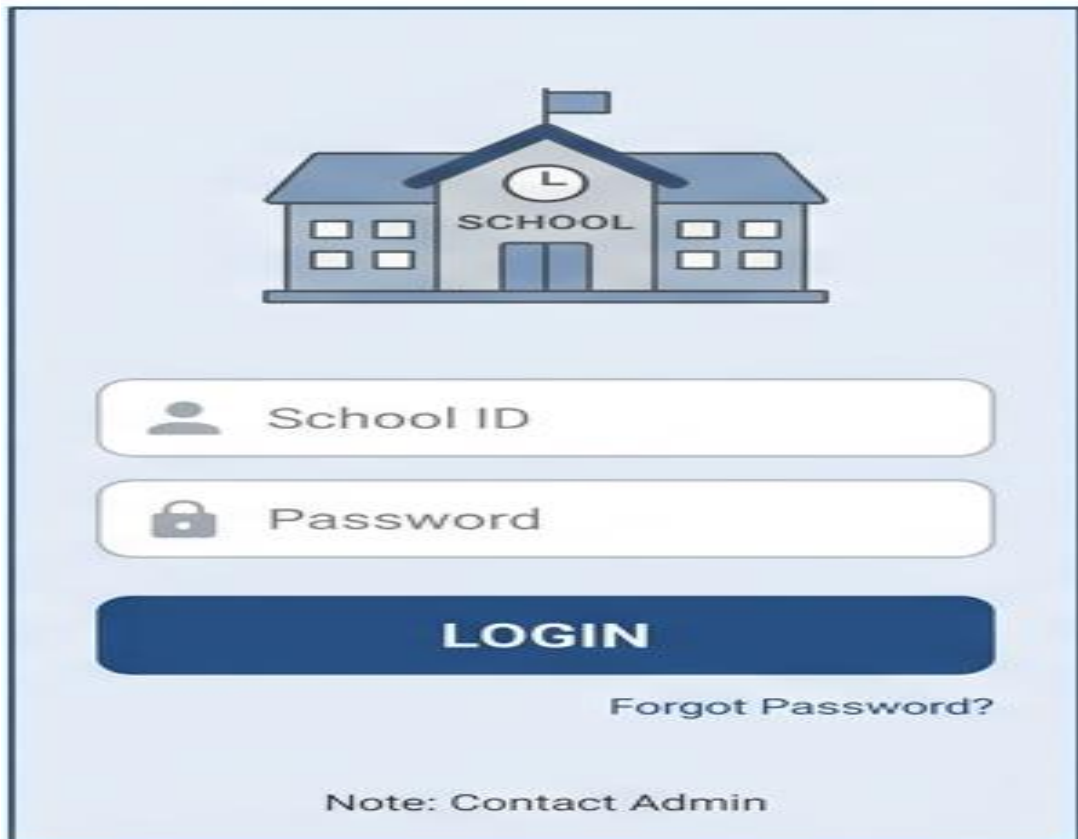


Figure 18. User Authentication Sequence Diagram



The login interface features a light blue background. At the top center is an illustration of a school building with a clock and the word 'SCHOOL' on its facade. Below this are two white input fields: the first is labeled 'School ID' with a person icon, and the second is labeled 'Password' with a lock icon. A large, dark blue 'LOGIN' button is positioned below the fields. To the right of the button is a link that says 'Forgot Password?'. At the bottom, a note reads 'Note: Contact Admin'.

Figure 19. User Login Interface



Figure 20. Student Home Dashbord



Figure 21. Interactive Library Seat Map



Figure 22. Seat Reservation Confirmation Dialog

FIGURE 22. SEAT RESERVATION CONFIRMATION DIALOG



Figure 22. Seat Reservation Confirmation Dialog

5. Interface description

The **Smart Library Management System (SLMS)** provides a user-friendly graphical interface designed for ease of seat reservation, real-time occupancy monitoring, automated check-ins, and generating analytics reports.

5.1. ModuleInterface

5.1.1. Student/User Module.

The Student Module is designed for effortless access to library facilities by students. It consists of various interfaces focused on personal booking management and profile customization.



Figure 23: My Reservations Page Interface This page displays a comprehensive list of the user's active and past reservations. Students may modify or cancel their bookings directly through this interface.



Figure 23. Admin My Reservations Page Interface

Figure 24: User Profile Update Form This form allows users to update personal information, such as email addresses and departments. It also includes preference settings for system notifications.

FIGURE 24. USER PROFILE UPDATE FORM



The image shows a tablet displaying a user profile update form. On the left side of the form, there is a circular placeholder for a profile picture with a 'CHANGE PHOTO' button below it. To the right of the profile picture area, there are five input fields labeled 'First Name', 'Last Name', 'Email Address', 'Email Address', and 'Profile photo'. At the bottom right of the form is a 'SAVE CHANGES' button.

Figure 24. User Profile Update Form

Figure 29: QR Code Check-in Interface Utilized for a rapid check-in process, students simply scan the QR code located at their assigned seat to confirm occupancy and start their session.

FIGURE 29. QR CODE CHECK-IN INTERFACE



The image shows a tablet displaying a QR code check-in interface. The screen features a large QR code in the center, framed by a white border. Below the QR code, the text 'Scan Code on Seat A12 to Check-in' is displayed. At the bottom of the screen, there is a green progress bar.

Figure 29.QR Code Check-in Interface

Figure 30: System Notifications & Alert Center This interface displays the log of all system alerts for the user, including reservation confirmations, reminders, and important management announcements.



Figure30.System Notifications & Alert Center

Figure 25: Admin Live Monitoring Dashboard This dashboard provides real-time occupancy statistics, featuring a gauge chart indicating the current percentage of capacity filled and bar graphs representing occupancy levels across various library zones.



Figure25.Admin Live Monitoring Dashboard

Figure 26: Admin Seat Management and Status Override Page This interface grants administrators the authority to manually override seat statuses, such as marking a specific seat unavailable for maintenance.



Figure26.Admin Live Monitoring Dashboard

FIGURE 26. ADMIN SEAT MANAGEMENT



Figure26.Admin Live Monitoring Dashboard

Figure 27: Library Capacity & Threshold Settings Page This settings console allows administrators to toggle features like "Automatic Seat Release" and "Overcrowding Alerts," as well as configure occupancy threshold limits.

FIGURE 27. LIBRARY CAPACITY & THRESHOLD SETTINGS



Figure27.Library Capacity & Threshold Settings Page

Figure 28: Analytics Report: Peak Hours & Occupancy Trends This interface presents historical data and occupancy trends to help library staff identify "Peak Hours"—specific times or days when the library reaches maximum capacity—to better manage student flow.



Figure28.Library Capacity & Threshold Settings Page

Conclusion and Summary of Design

The interface designs and system modules presented in this document establish a comprehensive framework for the **Smart Library Management System (SLMS)**. By integrating a user-centric mobile interface for students and a data-driven monitoring dashboard for administrators, the system effectively addresses the challenges of library overcrowding and inefficient seat management.

Each component, from the real-time **Interactive Seat Map** to the **Analytics Report on Occupancy Trends**, has been designed to ensure transparency, ease of use, and operational efficiency. The SLMS not only streamlines the reservation process but also provides the library administration with actionable insights to optimize the use of school facilities. Ultimately, this design serves as a robust blueprint for a modernized, technology-driven library environment that enhances the overall academic experience for the student body.